

Deep Image Restoration and Understanding Using Deep Learning

Kamyar PourMohammad

Contents

1	Project Overview	3
2	Project Pipeline	3
3	Phase 0: Environment Setup	4
3.1	Virtual Environment	4
3.2	Installing PyTorch	4
3.3	Installing Project Dependencies	4
3.4	Project Directory Structure	4
4	Phase 1: Deep Denoising	5
4.1	Pre-trained Model	5
5	Phase 2: Deep Deblurring	5
5.1	Why Is the Architecture Code Required?	6
5.2	ResNet Architecture	6
5.3	Model Weights	6
5.4	PSNR Evaluation	7
6	Phase 3: Image Classification	7
6.1	CIFAR-10 Dataset	7
6.2	Training the Classifier	7
7	Confusion Matrix Evaluation	8
8	Final Classification	8
9	Important Deep Learning Concepts	9
9.1	Pixel Normalization	9
9.2	Skip Connections	9
9.3	Training and Validation Loss	9
9.4	Accuracy and F1-Score	9
10	Project Deliverables	10

11 Analytical Questions	10
11.1 Question 1: Preprocessing Analysis	10
11.2 Question 2: Deblurring Architecture Analysis	10
11.3 Question 3: Learning Curve Analysis	11
11.4 Question 4: Evaluation Metrics	11
11.5 Question 5: Confusion Matrix Analysis	11
12 Conclusion	12

1 Project Overview

Throughout this course, we studied the fundamental concepts of image processing, spatial filters, frequency-domain analysis, and classical image enhancement techniques.

In this final project, we take a step further into the modern world of Deep Learning. The project is designed to simulate a realistic computer vision engineering scenario:

Given a heavily corrupted image, recover its visual and semantic information using deep neural networks.

The input image is affected by two major types of corruption:

1. **Salt-and-Pepper Noise:** Completely black and white pixels that destroy local image information.
2. **Blur:** The loss of sharp edges and high-frequency information.

The main objective is to process the corrupted image through a multi-stage deep learning pipeline and finally pass the restored image to a classifier.

The classifier will determine whether the restoration process was successful enough for an artificial intelligence model to understand the semantic content of the image.

2 Project Pipeline

The complete project pipeline is:

```
Corrupted Image
|
v
Phase 1: Deep Denoising
|
v
Denoised but Blurred Image
|
v
Phase 2: Deep Deblurring
|
v
Restored Image
|
v
Phase 3: Image Classification
|
v
Predicted Class + Confidence
```

3 Phase 0: Environment Setup

This project uses both PyTorch and TensorFlow.

Because image processing and deep learning require considerable computational resources, using a GPU is highly recommended.

If the local computer does not have a suitable GPU, Google Colab can be used as a cloud-based environment.

3.1 Virtual Environment

A Conda environment can be created using:

```
conda create -n dl_project_env python=3.10
conda activate dl_project_env
```

3.2 Installing PyTorch

The exact PyTorch installation command depends on the hardware and CUDA version.

For example:

```
conda install pytorch torchvision pytorch-cuda=11.8 -c pytorch -c nvidia
```

3.3 Installing Project Dependencies

The remaining dependencies can be installed using:

```
pip install -r requirements.txt
```

3.4 Project Directory Structure

The required directories are:

```
models/
noisy-inputes/
original-inputes/
phase_1_results/
phase_2_results/
utils/
models_architecture/
```

They can be created using:

```
python -c "import os; [os.makedirs(d, exist_ok=True) for d in ['models', 'noisy-inputes', 'original-inputes', 'phase_1_results', 'phase_2_results', 'utils', 'models_architecture']]"
```

4 Phase 1: Deep Denoising

The corrupted input image contains both Salt-and-Pepper noise and blur.

The objective of this phase is to remove the noise while preserving the overall structure of the image.

A pre-trained TensorFlow/Keras model is used for this stage.

4.1 Pre-trained Model

The pre-trained model is stored as:

```
denoising_model.h5
```

The model should be placed inside:

```
models/
```

The denoising function performs the following operations:

1. Loads the pre-trained model.
2. Resizes the input image to 256×256 pixels.
3. Normalizes pixel values to the range $[0, 1]$.
4. Adds the batch dimension.
5. Performs inference.
6. Converts the output back to 8-bit image format.

The function is called as follows:

```
denoising(  
image_path="...",  
dest_path="...",  
model_path="..."  
)
```

The input image should be located in:

```
noisy-inputs/
```

The output should be saved in:

```
phase_1_results/
```

The expected output is an image with significantly reduced noise but remaining blur.

5 Phase 2: Deep Deblurring

After the first phase, the image is mostly free of Salt-and-Pepper noise but still suffers from blur.

The objective of this phase is to recover sharp edges and high-frequency information. Three different architectures are provided:

1. CNN
2. ResNet
3. U-Net

For this project, the **ResNet architecture** is selected for the deblurring stage.

5.1 Why Is the Architecture Code Required?

TensorFlow/Keras models stored in the .h5 format often contain both the model architecture and the trained weights.

Therefore, the complete model can usually be loaded directly.

PyTorch models stored in the .pth format commonly contain only the trained parameters, known as a *state_{dict}*.

Therefore, the architecture must first be reconstructed in Python:

```
model = ResNet()
```

Then, the trained weights are loaded:

```
model.load_state_dict(...)
```

5.2 ResNet Architecture

ResNet uses shortcut connections, also known as skip connections.

These connections allow information to bypass one or more layers and travel directly through the network.

This is particularly useful for image restoration because fine details and edge information can be preserved during the restoration process.

5.3 Model Weights

The ResNet weights are stored in:

```
ResNet.pth
```

The model weights are placed inside:

```
models/ResNet.pth
```

The deblurring function is called as follows:

```
apply_deblur_model(  
    model_path="models/ResNet.pth",  
    input_path="phase_1_results/denoised_image.jpg",  
    destination_path="phase_2_results/deblurred_image.jpg",  
    model_name="resnet"  
)
```

The output of this phase is a sharper restored image.

5.4 PSNR Evaluation

The restored image is compared with the original clean image using the Peak Signal-to-Noise Ratio (PSNR).

The original image is used only as the Ground Truth.

The original image must not be provided to the neural networks during the restoration process.

A higher PSNR value generally indicates better similarity between the restored image and the original image.

6 Phase 3: Image Classification

The final phase evaluates whether the restoration pipeline successfully preserved the semantic content of the image.

A classifier is trained from scratch using the CIFAR-10 dataset.

6.1 CIFAR-10 Dataset

CIFAR-10 contains 60,000 color images of size 32×32 pixels.

The dataset contains ten classes:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

6.2 Training the Classifier

The classifier architecture is PreActResNet18.

The model is trained using:

```
train_model(  
    model_architect=PreActResNet18  
)
```

The trained model is saved as:

```
models/preact_resnet18_standalone.pt
```

The training process also produces training metrics such as:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

7 Confusion Matrix Evaluation

Before performing the final classification, the classifier is evaluated on the CIFAR-10 test set.

The confusion matrix is generated using:

```
plot_cifar_confusion_matrix(  
model_path="models/preact_resnet18_standalone.pt",  
data_dir="./data"  
)
```

The rows represent the actual classes, while the columns represent the predicted classes.

The main diagonal represents correct predictions.

Values outside the main diagonal represent classification errors.

8 Final Classification

The final restored image from Phase 2 is passed to the trained classifier.

For example:

```
classifier(  
model_path="models/preact_resnet18_standalone.pt",  
image_path="phase_2_results/deblurred_image.jpg",  
ground_truth="car"  
)
```

The classifier returns:

- The predicted class.
- The confidence score.

If the predicted class matches the ground-truth class, the restoration pipeline successfully preserved enough semantic information for the classifier to recognize the image.

9 Important Deep Learning Concepts

9.1 Pixel Normalization

Pixel values are commonly normalized from:

```
[ [0,255] ]
```

to:

```
[ [0,1] ]
```

using:

```
[ image.astype(np.float32) / 255.0 ]
```

This improves numerical stability and ensures that the input distribution is consistent with what the model encountered during training.

9.2 Skip Connections

Skip connections allow information to bypass intermediate layers.

They help preserve:

- Fine details
- Edges
- Spatial information

This is particularly important in image restoration tasks.

9.3 Training and Validation Loss

A model is generally learning properly when both training and validation loss decrease.

Overfitting may occur when:

- Training loss continues to decrease.
- Validation loss stops decreasing or begins increasing.

This indicates that the model may be memorizing the training data instead of learning general patterns.

9.4 Accuracy and F1-Score

Accuracy is defined as the percentage of correct predictions:

```
[ Accuracy =====
```

```
Correct Predictions / Total Predictions]
```

However, Accuracy can be misleading for highly imbalanced datasets.

The F1-Score combines Precision and Recall:

```
[ F1 ==
```

```
2 ×  $\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ ]
```

Therefore, F1-Score is often more informative when class distributions are highly imbalanced.

10 Project Deliverables

All required files should be compressed into a ZIP archive with the following naming format:

```
studentName_StudentNumber_Project.zip
```

The ZIP file should contain:

1. The final executed Jupyter Notebook.
2. The denoised image.
3. The deblurred image.
4. The training metrics plot.
5. The CIFAR-10 confusion matrix.
6. The final analytical report in PDF format.

Recommended output files include:

```
phase1_denoised.jpg  
phase2_deblurred.jpg  
PreActResNet_training_metrics.png  
cifar10_confusion_matrix.png  
project_report.pdf
```

11 Analytical Questions

11.1 Question 1: Preprocessing Analysis

If the following line were removed from the denoising phase:

```
input_tensor = image.astype(np.float32) / 255.0
```

what would happen to the model's prediction process?

Explain why removing pixel normalization would affect the model's performance.

11.2 Question 2: Deblurring Architecture Analysis

The PSNR value was calculated for the restored image during the deblurring phase.

Explain why an architecture such as ResNet can be more effective at restoring blurred edges than a simple neural network without shortcut connections.

Your explanation should be based on the role of skip connections in preserving information and fine details.

11.3 Question 3: Learning Curve Analysis

Analyze the training curves generated during the classification phase.

Determine whether the model suffered from overfitting or achieved a proper fit.

Support your answer by analyzing the behavior of:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

11.4 Question 4: Evaluation Metrics

Provide a real-world example, other than cancer detection, in which the dataset is highly imbalanced.

Explain how using Accuracy could be misleading in this situation.

Why would the F1-Score be a more appropriate evaluation metric?

11.5 Question 5: Confusion Matrix Analysis

Analyze the confusion matrix generated during the classification phase.

a)

Which two classes were most frequently confused with each other?

Identify the largest value outside the main diagonal of the confusion matrix.

b)

CNNs rely on visual features such as:

- Texture
- Shape
- Color
- Background information

Based on these visual features, explain why the network may have struggled to distinguish between the two classes identified in part (a).

What visual similarities between the two classes might have confused the network?

12 Conclusion

This project demonstrates a complete deep learning pipeline for image restoration and semantic understanding.

The pipeline combines:

1. Deep image denoising.
2. ResNet-based image deblurring.
3. Quantitative evaluation using PSNR.
4. Deep image classification using PreActResNet18.
5. Performance analysis using learning curves and a confusion matrix.

The project demonstrates how deep learning models can be combined to recover both the visual quality and semantic information of severely corrupted images.